

2. CONCEITOS BÁSICOS

- Existem vários **critérios de avaliação** de um algoritmo como: quantidade de trabalho requerido, quantidade de espaço requerido, simplicidade, exatidão de resposta e otimalidade.
- Este capítulo introduz os conceitos básicos de complexidade e **análise assintótica**, ilustrando

Visão geral do capítulo

- 2.1 Medidas de Complexidade.
Introduz as idéias de complexidade pessimista (pior caso) e complexidade média, bem como as medidas de complexidade de tempo e de espaço.
- 2.2 Critérios de Complexidade.
Introduz as idéias de operações fundamentais e de tamanho da entrada. Introduz também a idéia de comparação entre complexidades.
- 2.3 Ordens Assintóticas.
Introduz as idéias de comportamento assintótico (notações O , Ω e Θ), e examinam-se algumas de suas propriedades.
- 2.4 Exemplos.
Ilustram as idéias anteriores com alguns exemplos de algoritmos.

2.1 MEDIDAS DE COMPLEXIDADE

- A quantidade de trabalho requerido por um algoritmo não pode ser descrita simplesmente por um número, porque o número de operações básicas efetuadas em geral não é o mesmo para qualquer entrada. A expressão “quantidade de trabalho requerido” também é chamada *complexidade do algoritmo*.
- Se a complexidade é tomada como a máxima para qualquer entrada de um dado “tamanho”, a complexidade é chamada *complexidade no pior caso*. Se, entretanto, é levada em conta a probabilidade de ocorrência de cada entrada, a complexidade é chamada *complexidade média*.

2.1.1 Complexidade de Tempo

- Uma das medidas de complexidade mais importante é a *complexidade de tempo*.
- Existem vários *aspectos* a considerar na determinação da complexidade de tempo de um algoritmo. Um critério pode ser medir o tempo de execução, mas:
 - Uma pequena mudança no programa pode não representar uma mudança significativa no algoritmo, mas, apesar disso, a *velocidade de execução* pode ser afetada.
 - Além disso, se dois programas são comparados, primeiro numa máquina, depois em outra, as comparações podem dar *resultados diferentes*.
 - Este critério é fortemente dependente tanto da implementação quanto da máquina.
- Uma alternativa útil é uma *análise matemática* das dificuldades intrínsecas da resolução do problema computacionalmente.

2.1.2 Complexidade de Espaço

- A **memória** usada por um programa é dependente da implementação.
- Porém, algumas conclusões sobre o **espaço** podem ser tiradas examinando-se o algoritmo.
- Espaços como: a **área** para guardar as instruções, as constantes, as variáveis e os dados.
- Se os dados têm uma **representação óbvia** como uma matriz, são considerados para a análise de espaço somente os **espaços extras**.

2.2 CRITÉRIOS DE COMPLEXIDADE

- Para medir a quantidade de trabalho realizado por um algoritmo, é escolhida uma operação, chamada *operação fundamental*. Às vezes, é necessária mais de uma operação fundamental.
- Por exemplo, para um *algoritmo de ordenação*, uma operação fundamental natural é a *comparação entre elementos* quanto à ordem.

2.1.2 Complexidade de Algoritmo

- O tempo requerido por um algoritmo em dada entrada, pode ser medido através de suas *seqüências de execução*.
- Seja **A** um conjunto de algoritmos, **D** o conjunto de dados e **E** o conjunto de todas as seqüências de execuções de operações fundamentais.

• ***exec*** : $A \times D \rightarrow E$;

$exec(a, d) :=$ seqüência de execuções de operações fundamentais efetuadas na execução do algoritmo ***a***, com entrada ***d***.

• ***custo*** : $E \rightarrow \mathbb{IN}$;

$custo(s) :=$ comprimento da seqüência ***s***, definido conforme o peso estabelecido para as operações fundamentais.

• ***desemp***(***d***) := ***custo(exec(a, d))***;

O desempenho do algoritmo ***a***, na entrada ***d***, é dado por $custo(exec(a, d))$.

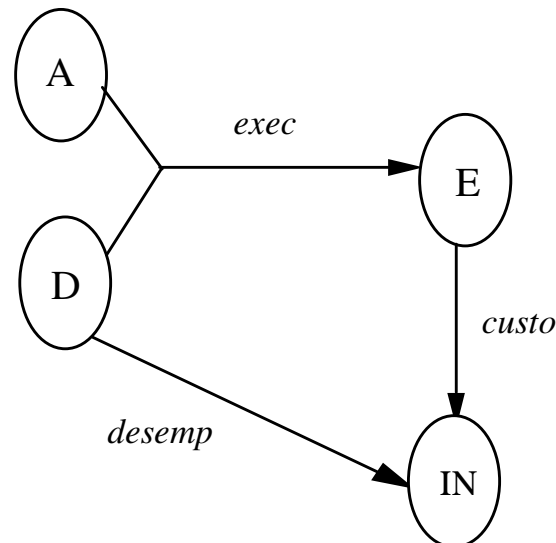


Figura 2.2.1: Desempenho do algoritmo

- Note que $desemp : D \rightarrow \mathbb{IN}$ dá informação sobre cada entrada específica.
- Deseja-se condensar a informação dada pela **função desempenho** em termos das entradas de um tamanho dado.
- Para tanto, seleciona-se uma função:
 θ **$tam : D \rightarrow \mathbb{IN}$** ;
 $tam(d) :=$ tamanho da entrada d , que mede o tamanho da entrada.
- De posse de tal função tam , pode-se condensar a informação dada pela função $desemp : D \rightarrow \mathbb{IN}$, em uma função **$aval : \mathbb{IN} \rightarrow \mathbb{IN}$** tal que:
 $aval(n) = desemp(d)$ para $tam(d) = n$

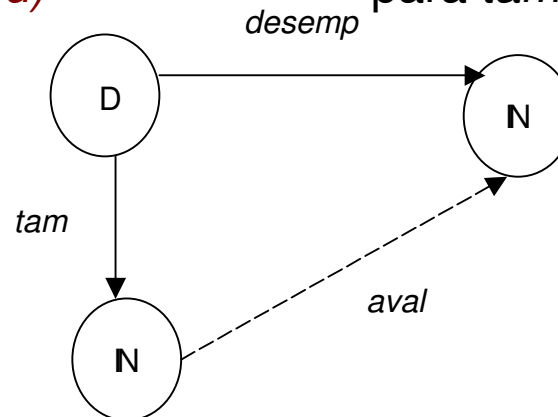


Figura 2.2.2: Avaliação de algoritmo

- Note que a função $tam : D \rightarrow \mathbb{N}$ em geral não é inversível. Costuma haver **várias entradas** com mesmo tamanho (e desempenhos distintos). **Qual delas escolher?**
- Dado $n \in \mathbb{N}$, seja: $D := \{ d / d \in D \text{ e } tam(d) = n \}$.
- Note que, em geral, a função $desemp$ não é constante sobre D . Assim, tomam-se **dois critérios**:
 - Complexidade pessimista (pior caso)
$$avalCP(n) := \text{Máx} \{ desemp(d) / d \in D \}$$
 - Complexidade média (caso médio)
$$avalCM(n) := \sum_{d \in D_n} \text{prob}(d) \cdot desemp(d) ,$$
onde $\text{prob}(d)$ dá a probabilidade de ocorrer a entrada d no conjunto D_n .

- Os critérios de complexidade que acabamos de introduzir consideram o **desempenho do algoritmo** sobre o conjunto de todas as entradas com tamanho n . Há um **critério alternativo**:

- No pior caso

$$c_P^{\bar{}}(n) = \text{Máx}\{desemp(d) / tam(d) = n\}$$

- No critério relaxado

$$c_P^{\leq}(n) = \text{Máx}\{desemp(d) / tam(d) \leq n\}$$

- O critério relaxado dá uma cota superior

$$c_P^{\bar{}}(n) \leq c_P^{\leq}(n) \quad \forall n \in \mathbb{N}$$

- A vantagem do critério relaxado é ser uma função monotônica

$$c_P^{\leq}(m) \leq c_P^{\leq}(n) \quad \text{para } m \leq n$$

2.2.2 Comparação entre complexidades

- Um problema pode ter **mais de um algoritmo** para resolvê-lo. **Qual deles escolher?**
- De maneira geral, consideremos um conjunto **A** de algoritmos para um dado problema Π .
- Escolhidas as operações fundamentais e a função tamanho, fixemos um **critério de complexidade**. Então, temos associado a cada algoritmo $a \in A$, sua avaliação $aval : \mathbb{N} \rightarrow \mathbb{N}$. Assim temos um funcional $A \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$.
- Frequentemente, interessa-nos identificar um **algoritmo ótimo**. Trata-se de um algoritmo $o \in A$, tal que $aval(o)$ é melhor do que $aval(a)$ para todo $a \in A$.
- Essas idéias envolvem comparações entre funções **f** e **g** de $\mathbb{N}$ em $\mathbb{N}$. Quando **f** é “**melhor**” do que **g**?
- Para isso, utiliza-se o conceito de comportamento **assintótico** (entradas de tamanho grande).

2.3 ORDENS ASSINTÓTICAS

- A **complexidade assintótica** é definida pelo crescimento da complexidade para entradas suficientemente grandes.
- O algoritmo assintoticamente **mais eficiente** é melhor para todas as entradas, exceto para entradas relativamente pequenas.
- Além disso, a complexidade exata costuma conter muita informação, resultando em uma função de **difícil análise** e **manipulação**.
- Consideremos funções **f** e **g**, mapeando naturais em reais não negativos: de $\mathbb{N}$ em $\mathbb{R}_+$.
- Uma **cota assintótica superior** (CAS) é uma função que cresce mais rapidamente do que outra: está acima, a partir de certo ponto.

- Em geral, define-se **g** uma cota assintótica superior para **f** sse:

$$(\exists n_0 \in \mathbb{N})(\forall n \geq n_0)(f(n) \leq g(n)),$$

significando que, para um n suficientemente grande, **g(n)** domina **f(n)**.

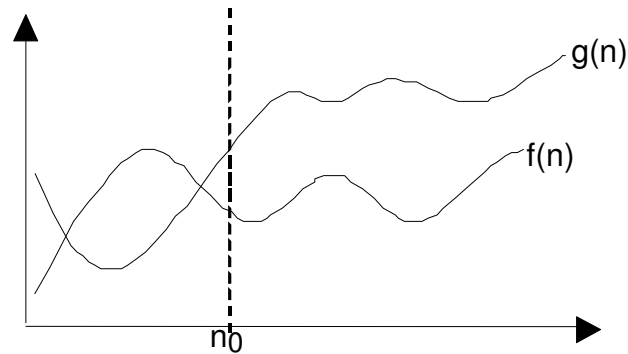


Figura 2.3.1: $g(n)$ é CAS para $f(n)$

Notação O

- A notação **O** define uma *cota assintótica superior*.
- Define-se que $f(n)$ é $O(g(n))$ sse:
para alguma constante $c \in \mathbb{R}_+$: **$c \cdot g(n)$ é CAS para $f(n)$** ,
i. e. $(\exists c \in \mathbb{R}_+) (\exists n_0 \in \mathbb{N}) (\forall n \geq n_0) (f(n) \leq c \cdot g(n))$.

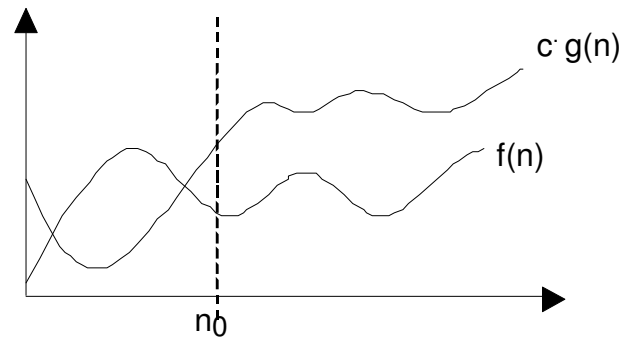


Figura 2.3.2: $f(n)$ é $O(g(n))$

Fato: Se $f(n) = a_m n^m + \dots + a_1 n + a_0$ é um polinômio de grau **m** então $|f(n)|$ é $O(n^m)$.

Demonstração:

$$\begin{aligned} |f(n)| &\leq |a_m| n^m + \dots + |a_1| n + |a_0| \\ &\leq (|a_m| + |a_{m-1}|/n + \dots + |a_0|/n^m) n^m \\ &\leq (|a_m| + |a_{m-1}| + \dots + |a_0|) n^m, \text{ para } n \geq 1. \end{aligned}$$

Então escolha $c = |a_m| + |a_{m-1}| + \dots + |a_0|$ e $n_0 = 1$ e tem-se:

$$|f(n)| \leq c n^m \text{ para todo } n \geq n_0.$$

Notação Ω

- A notação Ω define uma *cota assintótica inferior*.
- Define-se que $f(n)$ é $\Omega(g(n))$ sse:
para alguma constante $c \in \mathbb{R}_+$: $c \cdot f(n)$ é CAS para $g(n)$,
i. e. $(\exists c \in \mathbb{R}_+) (\exists n_0 \in \mathbb{N}) (\forall n \geq n_0) (g(n) \leq c \cdot f(n))$.

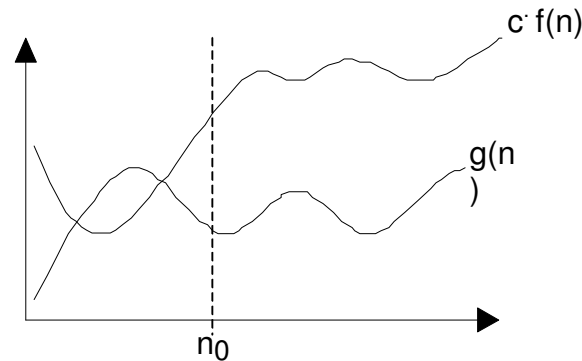


Figura 2.3.3: $f(n)$ é $\Omega(g(n))$

- Os algoritmos de $O(1)$, $O(n)$, $O(n^2)$, $\dots$ são *polinomiais*. Já 2^n é uma função não limitada por polinômio. Assim, um algoritmo de $\Omega(2^n)$, $\Omega(3^n)$, $\dots$ é dito *exponencial*.

Notação Θ

- A notação Θ define um *limite assintótico exato*.
- Diz-se que $f(n)$ é $\Theta(g(n))$.
- Define-se que $f(n)$ é $\Theta(g(n))$ sse:
 $(\exists c, c' \in \mathbb{R}_+) (\exists n_0 \in \mathbb{N}) (\forall n \geq n_0) (f(n) \geq c \cdot g(n) \wedge f(n) \leq c' \cdot g(n)).$

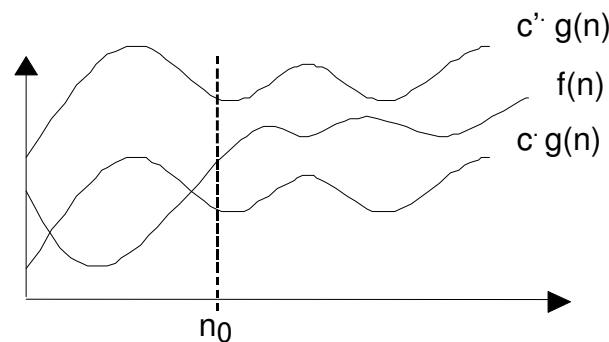


Figura 2.3.4: $f(n)$ é $\Theta(g(n))$

- Como $\log_a n = \Theta(\log_b n) \forall a, b$, para simplificar a notação muitas vezes, nos cálculos de complexidade será usado simplesmente $\log n$.

- Funções que são Θ uma da outra têm o mesmo comportamento assintótico.
- Note que a ordem Θ é um **caso particular** das ordens O e Ω , isto é, se $f(n)$ é $\Theta(g(n))$, então $f(n)$ também é $O(g(n))$ e $\Omega(g(n))$.
- Algumas **classes de comportamento assintótico** são listadas abaixo:

$$\begin{array}{ccc} \text{polinomiais} & & \text{exponenciais} \\ \underbrace{1 \quad \log n \quad n \quad n \log n \quad n^2 \quad n^3 \quad \dots} & \dots & \underbrace{2^n \quad 3^n \quad \dots} \end{array}$$

- Nessa lista, se $f(n)$ está à esquerda de $g(n)$, então $f(n) = O(g(n))$, e $g(n)$ não é $O(f(n))$.

2.4 EXEMPLOS

- **Exemplo 2.4.1:** O problema é determinar o valor máximo de um conjunto armazenado em uma tabela **tab** de **n** valores (reais, por exemplo).
 - *Algoritmo 2.1: Máximo de tabela.*

<u>Procedimento</u> Max(tab: tabela)	<i>{máximo de tabela}</i>
<i>{Entrada: tabela tab: vetor [1..n] de Elmord.</i>	
<i>{Saída: Mx = valor máximo em tab.}</i>	
1. $Mx \leftarrow tab[1];$	<i>{candidato a máximo}</i>
2. <u>para</u> i <u>de</u> 2 <u>até</u> n <u>faça</u>	<i>{varre tabela: de 2 a 4}</i>
3. <u>se</u> $Mx \leq tab[i]$ <u>então</u> $Mx \leftarrow tab[i];$	<i>{atualiza Mx}</i>
4. <u>fim-para</u> ;	<i>{2: i de 2 até n}</i>
5. <u>retorne-saída</u> (Mx);	<i>{fim do algoritmo: máximo de tabela}</i>
6. <u>fim-Procedimento</u> .	

- O algoritmo sempre tem **n-1** comparações, isto é $\Theta(n)$, como complexidade média no pior ou no melhor caso.

- **Exemplo 2.4.2:** O problema é procurar um valor (chave) em uma tabela **tab** para determinar se **ch** encontra-se ou não em **tab**.
 - *Algoritmo 2.2:* **Busca seqüencial em tabela.**

Procedimento BuscaSeq(ch: Valor, tab: tabela) {busca seqüencial em tabela}

{*Entrada*: ch: Valor, tabela tab: vetor [1..n] de Valor.

Saída: achou, pos (achou = F se ch não está em tab, senão achou = V e tab[pos] = ch).}

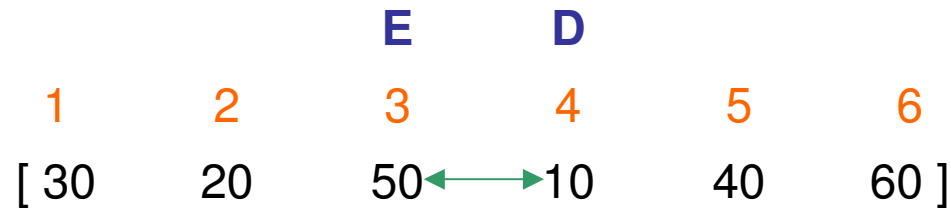
1. $i \leftarrow 0$; achou $\leftarrow$ F;	{inicialização}
2. <u>repita</u>	{varre tabela}
3. $i \leftarrow i+1$;	{incrementa índice}
4. <u>se</u> tab[i] = ch <u>então</u> achou $\leftarrow$ V;	{compara com chave}
5. <u>até</u> achou v $i = n$;	{fim do repita de 2}
6. <u>se</u> achou <u>então</u> pos $\leftarrow$ i;	{verifica se achou chave}
7. <u>retorne-saída</u> (achou, pos);	{fim do algoritmo: busca seqüencial em tabela}
8. <u>fim-Procedimento</u> .	

- **Operação fundamental**: a comparação = (no se da linha 4).
- A **relação** da complexidade com a posição de *ch* em *tab* é:
 - Se *tab* está:
 - na 1ª posição, BuscaSeq faz 1 comparação;
 - na 2ª posição, BuscaSeq faz 2 comparações;
 -;
 - na nª posição ou não está em *tab*, BuscaSeq faz n comparações.
- Complexidade no **pior caso**: $\Theta(n)$.
- Complexidade **média**: $\frac{1}{n} \cdot (1 + 2 + \dots + n) = \frac{1+n}{2}$ que é $\Theta(n)$.

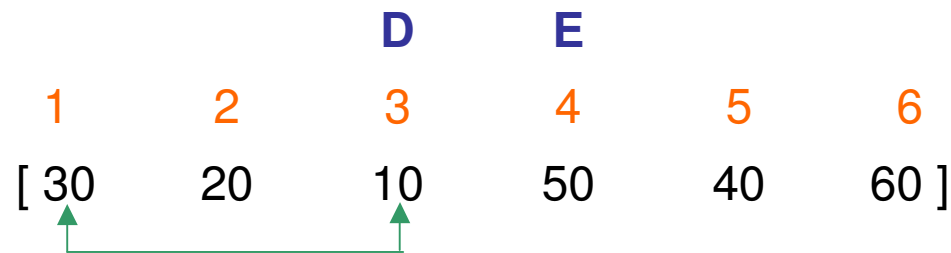
- **Exemplo 2.4.3:** O algoritmo é: dado um **pivô**, particionar os **elementos** das posições m até k de um vetor, movimentando-os de tal forma que no final, todos os elementos à direita do pivô serão maiores ou iguais a ele, enquanto os à sua esquerda serão menores que ele.
- O algoritmo usa dois apontadores que caminham em direções opostas:



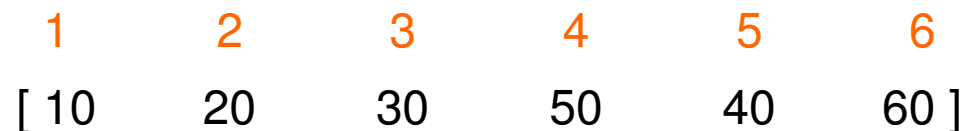
- Quando o apontador da esquerda encontra um elemento menor que o pivô e o apontador da direita, um elemento maior que o pivô, então eles são trocados:



- Quando o apontador da esquerda alcança ou ultrapassa o pivô da direita, o apontador é colocado na posição:



- Saída:



○ Algoritmo 2.3: **Particionamento**.

Procedimento Particionamento(A: vetor, m, k: Elmord) *{particionar vetor}*

{Entrada: vetor A: vetor [1..n] de Elmord, m, k.

Saída: (j, A), o vetor A com os elementos rearranjados de tal forma que todos os elementos inferiores a A[m] estarão à esquerda da posição j.}

```

1.  $v \leftarrow A[m]; i \leftarrow m+1; j \leftarrow k;$            {v é pivô; i apontador da esq.; j apontador da dir.}
2. repita                                           {2 a 6: percorre a nas duas direções}
3.   enqto  $A[i] < v$  faça  $i \leftarrow i+1;$            {elem. da esq. é < pivô, move apontador}
4.   enqto  $A[j] \geq v$  faça  $j \leftarrow j-1;$            {elem. da dir. é  $\geq$  pivô, move apontador}
5.   se  $i < j$  então troca( $A[i]$  e  $A[j]$ )           {A[i] maior e A[j] menor que pivô}
6.        $i \leftarrow i + 1;$ 
7.        $j \leftarrow j + 1;$ 
8.   fim-se;
9. até  $i > j;$                                            {2: todo a foi percorrido}
10.  $A[m] \leftarrow A[j]; A[j] \leftarrow v;$            {na posição j está o elem. < pivô mais à esquerda}
11. retorne-saída(j, A);                               {fim do algoritmo: particionar vetor}
12. fim-Procedimento.
```

2.5 RESUMO DO CAPÍTULO

- O objetivo deste capítulo é introduzir e ilustrar os conceitos básicos de complexidade (medidas e critérios) e de análise assintótica.
- As principais medidas de complexidade são tempo e espaço, relacionadas à velocidade e quantidade de memória. A complexidade depende da entrada particular: os principais critérios são pior caso e caso médio.
- A complexidade é determinada com base em operações fundamentais e no tamanho da entrada.
- Análise matemática fornece a complexidade intrínseca do algoritmo. A complexidade assintótica dá o desempenho para entradas de tamanho grande.
- As principais notações de comportamento assintótico dão limites superior (O), inferior (Ω) e exato (Θ), a menos de constantes multiplicativas.
- Algoritmos polinomiais têm complexidade $O(n^k)$; os com complexidade $\Omega(2^n)$ são exponenciais.